

Databases and SQL

SQL 2 -- CRUD 1

Presented By: Rehber Moin

Agenda

- More on Keys
 - Primary Keys (recap & deep dive)
 - Foreign Keys
 - Introduction to SQL
 - Create Databases & Tables
 - Syntax Deep-Dive
 - INSERT
 - SELECT
 - Sakila Database Overview
 - OLAP vs OLTP
-

Precap: Why Only One Primary Key?

The Election Analogy

Question: When contesting for elections in India, how many people are in the running for PM?

543 Members in the Lok Sabha -> Only **1 PM**.

The same logic applies to Primary Keys:

- There can be many **Candidate Keys** (just like many candidates).
- But only **one** is selected as the **Primary Key** (just like only one becomes PM).

Why?

MySQL (and most RDBMS) uses the Primary Key for critical internal operations:

1. **Disk Ordering** -- MySQL orders data on disk by the Primary Key.
2. **Default Return Order** -- By default, query results are returned based on key order.
3. **Default Index** -- A default index (clustered index) is automatically created on the Primary Key.

Since **sorting and indexing** needs to be done, and we can only physically sort data in **one** order on disk, we can only have **one** Primary Key.

Primary Key -- Recap

Only one of the Candidate Keys is selected as the Primary Key of the table.

Example

Name	Email	Ph No
------	-------	-------

Rahul	Rahul@mail.com	72xxxxx
Vikram	Vikram@mail.com	90xxxxx
Vanshika	Vanshika@mail.com	77xxxxx

Candidate Keys: {Email} , {Ph No} -- both can uniquely identify each row.

How to Choose a Good Primary Key

A good Primary Key should:

1. **Be fast to sort on** -- Integers are faster to compare/sort than strings.
2. **Have smaller size** -- Reduces indexing space. An INT (4 bytes) vs a VARCHAR email (20+ bytes).
3. **Not change** -- A stable key avoids expensive re-indexing.

Example: Email vs Phone No vs ID

Name	Email	Ph No
Rahul	Rahul@mail.com	72xxxxx
Vikram	Vikram@mail.com	90xxxxx
Vanshika	Vanshika@mail.com	77xxxxx

- **Between String and Integer, which is better?** Integer -- faster to sort, smaller storage.
- **Phone numbers can change.** Unstable. Would trigger re-indexing.

Best Practice: Add a dedicated **auto-incrementing integer ID** column as the Primary Key.

ID	Name	Email	Ph No
123	Rahul	Rahul@mail.com	72xxxxx
456	Vikram	Vikram@mail.com	90xxxxx
789	Vanshika	Vanshika@mail.com	77xxxxx

Foreign Keys

The Problem: Connecting Two Tables

Students Table:

S_ID	Name	PSP
123	Rahul	93
456	Vikram	95
789	Vanshika	90

Batches Table:

B_ID	Name
01	SQL
02	LLD
03	DSA

Question: Can you tell me which batch Rahul belongs to?

With no link between the tables, there's no way to answer this.

How to Connect Students & Batches?

Attempt 1: Add S_ID to Batches Table

B_ID	Name	S_ID
01	SQL	[123, 789]
02	LLD	456
03	DSA	

This **breaks atomicity** -- storing a list [123, 789] in one cell violates RDBMS rules.

Attempt 2 (Correct): Add B_ID to Students Table

S_ID	Name	PSP	B_ID
123	Rahul	93	01
456	Vikram	95	02
789	Vanshika	90	01

B_ID	Name
01	SQL
02	LLD
03	DSA

Now we can see: Rahul (B_ID=01) is in SQL, Vikram (B_ID=02) is in LLD, and Vanshika (B_ID=01) is also in SQL.

Foreign Key Definition

A **Foreign Key** is a column in a table that references a column (usually the Primary Key) in another table.

Note: The PK of one table is usually used as the FK in another.

Relationship Diagram:

Students (Child Table)

S_ID	Name	PSP	B_ID
123	Rahul	93	01
456	Vikram	95	02
789	Vanshika	90	01

Batches (Parent Table)

B_ID	Name
01	SQL
02	LLD
01	SQL

FK → PK

- **Parent Table:** batches (the table being referenced -- owns the PK).
- **Child Table:** students (the table that holds the FK).

Foreign Keys -- Maintaining the Relationship

"Creating a Relationship is Easy, But Maintaining it is Difficult."

Problem 1: What Happens on UPDATE?

Suppose Scaler decides that the B_ID for LLD is updated from 02 to 04 :

Batches Table (after update):

B_ID	Name
01	SQL
04	LLD
03	DSA

Students Table (unchanged):

S_ID	Name	PSP	B_ID
123	Rahul	93	01
456	Vikram	95	02
789	Vanshika	90	01

Vikram's B_ID is still 02 , but batch 02 no longer exists. **Data is now inconsistent.**

Problem 2: What Happens on DELETE?

Suppose Scaler decides to delete the LLD batch entirely:

Batches Table (after delete):

B_ID	Name
01	SQL
--	--

Vikram's record still references `B_ID = 02`, but that batch is gone. The child entry becomes **orphaned**. Still inconsistency.

The Fix

*When updating or deleting in the Parent table, **update the child records/entries as well**.*

This is called **Referential Integrity**: The Foreign Key must ensure that the parent-child relationship remains consistent.

To ensure Referential Integrity, we define **ON UPDATE** and **ON DELETE** rules.

ON UPDATE / ON DELETE Strategies

Strategy	On UPDATE	On DELETE	Supported in MySQL?
CASCADE	Child FK values are automatically updated to match	Child rows are automatically deleted	Yes
SET NULL	Child FK values are set to NULL	Child FK values are set to NULL	Yes
NO ACTION	Update/Delete is blocked if child rows exist	Update/Delete is blocked if child rows exist	Yes (Default)
SET DEFAULT	Child FK values are set to a default value	Child FK values are set to a default value	No (Not supported in MySQL)

CASCADE Example (ON UPDATE)

Starting state:

Students

S_ID	Name	PSP	B_ID
123	Rahul	93	01
456	Vikram	95	02
789	Vanshika	90	01

Batches

B_ID	Name
01	SQL
02	LLD
03	DSA

If we update `B_ID` from `02` to `04` in Batches (with `ON UPDATE CASCADE`):

Students

S_ID	Name	PSP	B_ID
123	Rahul	93	01

Batches

B_ID	Name
01	SQL

456	Vikram	95	04	04	LLD	<-- Updated
789	Vanshika	90	01	03	DSA	

^
Automatically updated from 02 to 04

SET NULL Example (ON DELETE)

If we delete the LLD batch (B_ID=02) from Batches (with `ON DELETE SET NULL`):

Students				Batches	
S_ID	Name	PSP	B_ID	B_ID	Name
123	Rahul	93	01	01	SQL
456	Vikram	95	NULL	03	DSA
789	Vanshika	90	01		

^
Set to NULL (LLD batch was deleted)

NO ACTION / RESTRICT (Default in MySQL)

If no `ON UPDATE` or `ON DELETE` rule is specified, MySQL uses **NO ACTION** (also called **RESTRICT**):

- The database **blocks** any UPDATE or DELETE on the parent table if child rows reference the affected value.
- You must first remove or update the child references before modifying the parent.

Introduction to SQL

SQL = Structured Query Language

SQL is the standard language used to interact with relational databases.

CRUD Operations

All operations on a database boil down to four types:

Operation	SQL Command(s)	Description
Create	CREATE, INSERT	Create databases, tables, rows
Read	SELECT	Query and retrieve data
Update	UPDATE, ALTER	Modify existing data/schema
Delete	DELETE, DROP	Remove data or objects

SQL Syntax Deep-Dive

Comments

```
-- This is a single-line SQL comment
```

Show All Databases

```
SHOW DATABASES;
```

Create a Database

```
CREATE DATABASE scaler;
```

Select a Database to Use

Before creating or querying tables, you must specify which database to work in:

```
USE scaler;
```

CREATE TABLE

```
CREATE TABLE students (  
  id      INT PRIMARY KEY AUTO_INCREMENT,  
  name    VARCHAR(50) NOT NULL,  
  psp     INT  
);
```

Key Points:

Keyword	Meaning
INT	Integer data type
PRIMARY KEY	Marks this column as the Primary Key
AUTO_INCREMENT	MySQL automatically generates the next integer value
VARCHAR(50)	Variable-length string up to 50 characters
NOT NULL	This column cannot be empty -- a value must always be provided

CHAR vs VARCHAR

Type	Storage	Use Case
CHAR(50)	Always uses 50 bytes	Fixed-length data (e.g., country codes)
VARCHAR(50)	Uses only as many bytes as needed (+ 1-2 bytes overhead)	Variable-length data (e.g., names)

`VARCHAR` is more **space-efficient** for columns where data length varies.

Read Data from a Table

```
SELECT * FROM students;
-- '*' means select ALL columns from the table
```

INSERT INTO

Method 1: Without Column Names (Not Recommended)

```
INSERT INTO students VALUES(DEFAULT, 'Schumacher', 87);
```

- You must provide values for **all** columns in the correct order.
- Use `DEFAULT` for auto-increment columns.
- **Risky** for larger, more complicated tables -- if column order changes, the query breaks.

```
-- This will FAIL -- MySQL doesn't know what maps to what:
INSERT INTO students VALUES('Schumacher', 87);
```

Method 2: With Column Names (Recommended)

```
INSERT INTO students (name, psp)
VALUES ('Nikki', 90);
```

- You explicitly state which columns you're inserting into.
- No ambiguity, safe even if columns are reordered.
- Columns with defaults or `AUTO_INCREMENT` can be omitted.

What if You Don't Provide PSP?

```
INSERT INTO students (name)
VALUES ('Senna');
```

PSP will be **NULL** (or the default value if one was defined).

You can set a default during table creation:

```
psp INT DEFAULT 80
```

NULL in SQL

`NULL` in SQL is **not** the same as `null` in most programming languages. In SQL, `NULL` represents an **unknown or missing value**. It is not limited to strings -- any data type can be `NULL`.

Inserting Multiple Rows

```
INSERT INTO students (name, psp)
VALUES
  ('Schumacher', 87),
  ('Nikki', 90),
  ('Verstappen', 92);
```

SELECT Statements

Select Specific Columns

```
-- Only get student names
SELECT name FROM students;
```

Aliasing with AS

You can rename columns in the result set using `AS` (this does **not** change the actual column name in the table):

```
SELECT name AS 'Full Name' FROM students;
```

Foreign Keys in SQL

Step 1: Create the Parent Table First

```
CREATE TABLE batches (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50)
);

INSERT INTO batches (name)
VALUES ('LLD'), ('SQL'), ('HLD');

SELECT * FROM batches;
```

Result:

```
+----+-----+
| id | name |
+----+-----+
|  1 | LLD  |
|  2 | SQL  |
|  3 | HLD  |
+----+-----+
```

Step 2: Create the Child Table with a Foreign Key

```
CREATE TABLE students (
  id      INT PRIMARY KEY AUTO_INCREMENT,
  name    VARCHAR(50) NOT NULL,
  psp     INT DEFAULT 80,
  batch_id INT,
  FOREIGN KEY (batch_id) REFERENCES batches(id)
);
```

What this means:

```
students.batch_id —references—> batches.id
(Foreign Key)                (Primary Key)
```

Step 3: Insert Data into Students

```
INSERT INTO students (name, psp, batch_id)
VALUES
  ('Schumacher', 87, 1),
  ('Nikki', 90, 1),
  ('Verstappen', 92, 2),
  ('Hulkenberg', 87, 1),
  ('Charles', 87, 3);

SELECT * FROM students;
```

Result:

id	name	psp	batch_id
1	Schumacher	87	1
2	Nikki	90	1
3	Verstappen	92	2
4	Hulkenberg	87	1
5	Charles	87	3

Default Behaviour (No ON UPDATE / ON DELETE Specified)

When no rules are specified, the default is **RESTRICT / NO ACTION**:

- **Updating** a B_ID in batches that is referenced by students will be **blocked**.
- **Deleting** a batch that is referenced by students will be **blocked**.
- The only time MySQL allows the update or delete is when **no child rows reference** that value.

Step 4: Recreate with ON UPDATE CASCADE and ON DELETE SET NULL

```
DROP TABLE students;
DROP TABLE batches;
```

```

CREATE TABLE batches (
  id    INT PRIMARY KEY AUTO_INCREMENT,
  name  VARCHAR(50)
);

INSERT INTO batches (name)
VALUES ('LLD'), ('SQL'), ('HLD');

CREATE TABLE students (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  name        VARCHAR(50) NOT NULL,
  psp         INT DEFAULT 80,
  batch_id    INT,
  FOREIGN KEY (batch_id) REFERENCES batches(id)
    ON UPDATE CASCADE
    ON DELETE SET NULL
);

INSERT INTO students (name, psp, batch_id)
VALUES
  ('Schumacher', 87, 1),
  ('Nikki', 90, 1),
  ('Verstappen', 92, 2),
  ('Hulkenberg', 87, 1),
  ('Charles', 87, 3);

```

Testing ON UPDATE CASCADE

```

-- Before: batches
SELECT * FROM batches;
-- +-----+-----+
-- | id | name |
-- +-----+-----+
-- | 1 | LLD |
-- | 2 | SQL |
-- | 3 | HLD |
-- +-----+-----+

-- Update batch id 1 -> 10
UPDATE batches SET id = 10 WHERE id = 1;

-- After: batches
SELECT * FROM batches;
-- +-----+-----+
-- | id | name |
-- +-----+-----+
-- | 10 | LLD |    <-- changed from 1 to 10
-- | 2 | SQL |
-- | 3 | HLD |
-- +-----+-----+

```

```

-- After: students (CASCADE automatically updated batch_id)
SELECT * FROM students;
-- +-----+-----+-----+-----+
-- | id | name      | psp | batch_id |
-- +-----+-----+-----+-----+
-- | 1 | Schumacher | 87 | 10 | <-- was 1, now 10
-- | 2 | Nikki      | 90 | 10 | <-- was 1, now 10
-- | 3 | Verstappen | 92 | 2 |
-- | 4 | Hulkenberg | 87 | 10 | <-- was 1, now 10
-- | 5 | Charles    | 87 | 3 |
-- +-----+-----+-----+-----+

```

Testing ON DELETE SET NULL

```

-- Delete batch with id = 2 (SQL batch)
DELETE FROM batches WHERE id = 2;

-- After: batches
SELECT * FROM batches;
-- +-----+-----+
-- | id | name |
-- +-----+-----+
-- | 10 | LLD  |
-- | 3  | HLD  |
-- +-----+-----+

-- After: students (SET NULL automatically set batch_id to NULL)
SELECT * FROM students;
-- +-----+-----+-----+-----+
-- | id | name      | psp | batch_id |
-- +-----+-----+-----+-----+
-- | 1 | Schumacher | 87 | 10 |
-- | 2 | Nikki      | 90 | 10 |
-- | 3 | Verstappen | 92 | NULL | <-- was 2, now NULL
-- | 4 | Hulkenberg | 87 | 10 |
-- | 5 | Charles    | 87 | 3 |
-- +-----+-----+-----+-----+

```

OLAP vs OLTP

Feature	OLTP	OLAP
Full Form	Online Transaction Processing	Online Analytical Processing
Purpose	Day-to-day transactional operations	Complex analytical queries & reporting
Queries	Simple, fast (INSERT, UPDATE, SELECT by ID)	Complex (aggregations, joins across large data)

Data	Current, up-to-date	Historical, aggregated
Examples	Banking transactions, e-commerce orders	Business intelligence, data warehousing
Optimized	Write-heavy workloads	Read-heavy workloads

Summary

Topic	Key Takeaway
Primary Key	Only one per table. Used for disk ordering, indexing, and sorting.
Choosing a Good PK	Should be fast to sort, small in size, and stable (unlikely to change). Use auto-increment INT.
Foreign Key	A column referencing another table's PK. Links parent and child tables.
Referential Integrity	FK must ensure parent-child consistency on updates and deletes.
ON UPDATE CASCADE	Child FK values auto-update when parent PK changes.
ON DELETE SET NULL	Child FK values become NULL when parent row is deleted.
NO ACTION (Default)	Blocks updates/deletes on parent if child references exist.
CRUD	Create, Read, Update, Delete -- the four fundamental DB operations.
INSERT INTO	Always use column names for safety.
SELECT	Use * for all columns, or list specific columns. Use AS for aliasing.